🔑 Key Differences:

Sequential Pipeline (Strategy 1):

Input → [NER] → [Mapping] → [Validation] → [Generation] → Output

Step 1 Step 2 Step 3 Step 4

- **4 separate components**
- Errors compound: 5% + 10% + 5% + 5% ≈ **23% total failure**
- ✅ Easy to debug
- ✅ Good generalization
- ❌ Complex to maintain

Joint Training (Strategy 2):

(Description, Formula) pairs → [Single Model] → Output

- **1 end-to-end model**
- No error propagation between steps
- ✅ Simple architecture
- ✅ 100% accuracy on seen patterns
- ❌ Fails completely on unseen patterns
- ❌ Hard to debug

📊 Performance Comparison:

Metric	Sequential	Joint
Seen Patterns	~85-90%	~100% ✅
Unseen Patterns	~75% ✅	~0% ❌
Debugging	Easy ✅	Hard ❌
Speed	Slower	Faster ✅
Maintenance	Complex	Simple ✅

💡 Best Practice:

Use Both (Hybrid Approach):

python

```
def smart_predict(description):
    # 1. Try joint model (fast, accurate for known patterns)
    result = joint_model.predict(description)
```

```
    if result == "Unknown pattern":
        # 2. Fallback to sequential (handles new cases)
        result = sequential_pipeline.process(description)

    return result
```

This gives you:

- ✅ Speed for common cases (joint)
- ✅ Robustness for edge cases (sequential)
- ✅ Best of both worlds

The flowchart artifact shows all this visually with interactive hover effects! 🎨